SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA1407 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks:50**

Annexure A

DATA INTERPRETATION

Code of Conduct:

I M.VINAY(192311146)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

M.VINAY
Signature of the student:

DATA INTERPRETATION

Q1. Intermediate Code Analysis and Optimization
Given the three-address code:
t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p
Analyze the sequence of operations and construct the equivalent expression tree. Evaluate how temporary variables improve intermediate code generation and reduce repeated computations.

Q2. Symbol Table and Memory Allocation
Given the following symbol table entries:

Variable	Data Type	Size (Bytes)
p	char	1
q	double	8
r	int	4
s	float	4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

Q3. Optimization in Intermediate Code
Given the intermediate code:
t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3
Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

Q4. Control Flow and Boolean Expression Handling
Given the Boolean expression:
if (x > y || p < q)
Intermediate code:
if x > y goto L1
if p < q goto L1
goto L2
L1: ...
Interpret the control flow of the given intermediate representation and explain how short-circuit evaluation minimizes unnecessary condition checking during execution.

Q5. Backpatching and Flow Control
Given:

Instruction	Target
200: if x==y goto _	

Instruction Target

201: goto _

Backpatch lists:

- truelist = {200}
- falselist = {201}

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

ANSWERS:

Q1. Intermediate Code Analysis and Optimization

Given:
t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p

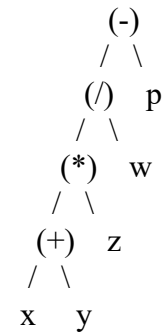
1. Sequence of operations

The operations are performed in the following order:

1. t1 = x + y
2. t2 = t1 * z
3. t3 = t2 / w
4. t4 = t3 - p

Therefore, the equivalent expression is:
((x + y) * z / w) - p

2. Equivalent Expression Tree



3. Role of Temporary Variables

Temporary variables such as t1, t2, t3, and t4 store intermediate results.
For example:
t1 = x + y
stores x + y, which can then be used directly in the next operation.
Advantages:

- Breaks complex expressions into simple operations.
- Makes code generation easier for the compiler.
- Provides explicit intermediate results.
- Avoids recalculating already computed expressions.
- Makes optimization easier.
- Helps map high-level expressions to machine instructions.

Thus, temporary variables improve the **clarity, efficiency, and manageability** of intermediate code.

Q2. Symbol Table and Memory Allocation

Given:

Variable	Data Type	Size
p	char	1 byte
q	double	8 bytes
r	int	4 bytes
s	float	4 bytes

1. Memory Allocation

Assuming variables are allocated sequentially without considering padding:
p → 1 byte
q → 8 bytes

r → 4 bytes
s → 4 bytes
Total:
Total = 1 + 8 + 4 + 4
= 17 bytes
Therefore, the minimum memory required is **17 bytes**.

2. Effect of Alignment

Modern processors generally require certain data types to start at suitable memory addresses.
For example:

- char → usually 1-byte alignment
- int → usually 4-byte alignment
- float → usually 4-byte alignment
- double → commonly 8-byte alignment

If p starts at address 1000:
p → 1000 (1 byte)

Padding → 1001–1007

q → 1008–1015 (8 bytes)

r → 1016–1019 (4 bytes)

s → 1020–1023 (4 bytes)
In this example, **7 bytes of padding** are introduced before q.
So the total allocated memory can become:
17 + 7 = 24 bytes

3. Effect of Data Types

Different data types require different amounts of memory because they represent different ranges and precision.
For example:

- char requires less memory.
- int is used for integer values.
- float provides single-precision floating-point storage.
- double provides greater precision and generally requires more memory.

Therefore, **data types determine storage requirements, while alignment can introduce padding and increase the actual memory allocated**.

Q3. Optimization in Intermediate Code

Given:
t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3

1. Step-by-Step Execution

Step 1:
t1 = m * n
The product of m and n is calculated and stored in t1.

Step 2:
t2 = p + q
The sum of p and q is calculated and stored in t2.

Step 3:
t3 = m * n
The same expression m * n is calculated again.
This is redundant because it was already calculated in Step 1.
Step 4:
t4 = t2 - t3
The value of t3 is subtracted from t2.
2. Optimization Opportunity
The expression:
m * n
appears twice:
t1 = m * n
t3 = m * n
This is a **common subexpression**.
We can reuse t1:
t1 = m * n
t2 = p + q
t3 = t1
t4 = t2 - t3
Or, because t3 is only used once:
t1 = m * n
t2 = p + q
t4 = t2 - t1
3. Benefits of Optimization
Optimization:

- Eliminates repeated multiplication.
- Reduces the number of instructions.
- Reduces CPU operations.
- Improves execution speed.
- Can reduce temporary storage requirements.

Thus, **common subexpression elimination** improves the efficiency of intermediate code by reusing previously calculated results.

Q4. Control Flow and Boolean Expression Handling
Boolean expression:
if (x > y || p < q)
Intermediate code:
if x > y goto L1
if p < q goto L1
goto L2

L1: ...
1. Control Flow
The first condition is checked:
if x > y goto L1
If:
x > y = TRUE
the control immediately jumps to L1.
Therefore, p < q does **not** need to be checked.
If:

x > y = FALSE
execution continues to:
if p < q goto L1
If p < q is true, execution goes to L1.
If both conditions are false:
goto L2
So the control flow is:

```
      x > y?
     /    \
   TRUE    FALSE
   |        |
   L1      p < q?
          /   \
        TRUE   FALSE
         |     |
         L1    L2
```

2. Short-Circuit Evaluation
The OR (||) operator uses short-circuit evaluation.
For:
A || B
if A is true, the complete expression is already true, so B is not evaluated.
Here:
x > y || p < q
If x > y is true, the compiler immediately jumps to L1.
This:

- Avoids unnecessary condition checking.
- Reduces execution time.
- Can prevent evaluation of expensive expressions.
- Produces efficient control-flow code.

Therefore, short-circuit evaluation improves the **performance of Boolean expression execution**.

Q5. Backpatching and Flow Control
Given:

Instruction	**Target**
200: if x == y goto	_
201: goto	_

Backpatch lists:
truelist = {200}
falselist = {201}
1. Role of Backpatching
Backpatching is a technique used by the compiler to temporarily leave jump targets blank and fill them when the correct target address becomes known.
Initially:
200: if x == y goto _
201: goto _
The compiler does not yet know where the true and false branches should go.
The lists record the incomplete instructions:
truelist = {200}
falselist = {201}

2. Updating the True Target
Suppose the true branch begins at instruction 205.
The compiler backpatches instruction 200:
200: if x == y goto 205
So:
truelist = {200}
is updated with target 205.
3. Updating the False Target
Suppose the false branch begins at instruction 210.
Instruction 201 is updated:
201: goto 210
Therefore:
falselist = {201}
is backpatched with target 210.
4. Final Intermediate Code
200: if x == y goto 205
201: goto 210

205: ... TRUE block ...
...
209: goto 215

210: ... FALSE block ...
...

215: ... NEXT statement ...
5. Importance of Backpatching
Backpatching is important because the compiler may generate jump instructions **before the final target addresses are known**.
It is particularly useful for:

- Boolean expressions
- if-else statements
- while loops
- for loops
- Conditional jumps
- Complex control-flow structures

Thus, backpatching provides an efficient way to generate and complete **control-flow information during intermediate code generation**.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)

Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand